

KIV/ZOS - INodes filesystem

Michal Malík

16. ledna 2026

Obsah

1	Úvod	1
1.1	Klíčové vlastnosti systému	1
2	Instalace a sestavení	2
2.1	Požadavky	2
2.2	Struktura projektu	2
2.3	Kompilace	2
2.4	Spuštění a ovládání	2
3	Architektura systému	3
3.1	Rozložení disku	3
3.2	Struktura Inody (Inode)	3
3.3	Adresářová struktura	3
4	Uživatelská příručka (Příkazy)	4
4.1	Správa disku a systému	4
4.2	Práce se soubory	4
4.3	Práce s adresáři	4
4.4	Import a Export (Hostitelský systém)	4
5	Příklad použití	5
6	Závěr	6

1 Úvod

Tato semestrální práce se zabývá návrhem a implementací simulátoru unixového souborového systému v jazyce C++. Aplikace nespravuje fyzický pevný disk, ale vytváří tzv. *virtuální disk*, binární soubor na hostitelském operačním systému, uvnitř kterého simuluje blokové zařízení.

Cílem projektu není pouze emulovat příkazovou řádku, ale věrně simulovat vnitřní mechanismy souborových systémů na styl I-uzlů. Projekt řeší problematiku perzistentního ukládání dat, což znamená, že stav systému (adresářová struktura, soubory, metadata) zůstává zachován i po ukončení aplikace.

1.1 Klíčové vlastnosti systému

Implementované řešení pokrývá následující oblasti správy dat:

- **Virtuální blokové zařízení:** Abstrakce pevného disku o libovolné velikosti (např. 100 MB), rozděleného na logické bloky o velikosti 4 KB.
- **Správa volného místa:** Využití bitových map (Bitmaps) pro efektivní $O(1)$ alokaci a uvolňování datových bloků a inodů.
- **Hierarchický systém adresářů:** Podpora zanořených složek, absolutních i relativních cest.
- **Adresace velkých souborů:** Implementace přímých (Direct), nepřímých (Single Indirect) a dvojitě nepřímých (Double Indirect) odkazů, umožňující správu větších souborů.
- **Transakční bezpečnost:** Mechanismy zabráňující poškození souborového systému v případě nedostatku místa (rollback operace).
- **Práce s hostovacím zařízením:** Možnost importu a exportu souborů mezi virtuálním diskem a hostitelským operačním systémem.

2 Instalace a sestavení

Projekt je navržen jako konzolová aplikace s minimálními závislostmi, což zajišťuje snadnou přenositelnost mezi systémy Linux, macOS a Windows (s využitím MinGW nebo WSL).

2.1 Požadavky

Pro úspěšné sestavení a běh aplikace jsou vyžadovány následující nástroje:

- Překladač jazyka C++ podporující standard **C++17** (např. **g++** verze 7.0+, **clang** nebo **MSVC**).
- Základní standardní knihovna C++ (STL).

2.2 Struktura projektu

Zdrojový kód je rozdělen do modulů pro lepší čitelnost a údržbu:

- `main.cpp` – Vstupní bod aplikace, smyčka příkazové řádky (REPL).
- `filesystem.cpp` / `.h` – Hlavní logika souborového systému (operace s inodami, bloky).
- `disk.cpp` / `.h` – Funkce pro správu virtuálního disku
- `inode.cpp` / `.h` – Struktura i-uzlu a pomocné funkce pro manipulaci.
- `utils.cpp` / `.h` – Pomocné funkce pro parsování cest a validaci.
- `directory.h` – Definice struktur pro adresářové záznamy.

2.3 Kompilace

Pro sestavení projektu otevřete terminál v kořenovém adresáři projektu a spusťte následující příkaz:

```
1 g++ -std=c++17 -Wall -Wextra -o MyNodes main.cpp filesystem.cpp inode.cpp  
    utils.cpp
```

Nebo můžete použít připravený Makefile příkazem `make all`

2.4 Spuštění a ovládání

Spustí aplikaci a načte virtuální disk. Uživatel zadává příkazy ručně. Pokud disk ještě neexistuje, nebo nemá standardní formát, uživateli je nabídnuto naformátovat nový soubor, aby odpovídal struktuře systému.

```
1 ./MyNodes filesystem.dat
```

3 Architektura systému

Souborový systém je rozdělen do několika logických sekcí uvnitř binárního souboru. Velikost bloku je fixně nastavena na **4096 bajtů (4KB)**.

3.1 Rozložení disku

1. **Superblok (Block 0):** Obsahuje globální informace o souborovém systému (celková velikost, počet volných inodů/bloků, magic number).
2. **Inode Bitmap (Block 1):** Bitová mapa sledující obsazenost inodů.
3. **Data Bitmap (Block 2):** Bitová mapa sledující obsazenost datových bloků.
4. **Tabulka Inodů (Blocks 3+):** Pole struktur Inode, které obsahují metadata o souborech.
5. **Datové bloky:** Zbytek disku slouží pro uložení obsahu souborů a adresářů.

3.2 Struktura Inody (Inode)

Každý soubor nebo adresář je reprezentován jednou inodou o velikosti 64 bajtů. Inoda obsahuje:

- **ID a Typ:** Unikátní číslo a příznak, zda jde o soubor či adresář.
- **Velikost:** Aktuální velikost souboru v bajtech.
- **Časová razítka:** Čas vytvoření a poslední modifikace.
- **Adresace dat:**
 - **Direct Blocks (10x):** Přímé odkazy na prvních 40KB dat.
 - **Single Indirect (1x):** Odkaz na tabulku s 1024 odkazy (kapacita +4MB).
 - **Double Indirect (1x):** Odkaz na „master“ tabulku odkazující na další tabulky (kapacita +4GB).

3.3 Adresářová struktura

Adresáře jsou implementovány jako speciální soubory, jejichž obsahem je seznam položek typu `DirEntry`. Každý záznam má fixní velikost 16 bajtů:

- **Název souboru:** Max. 12 znaků (11 znaků + null terminator).
- **Inode ID:** 4bajtové celé číslo odkazující na inodu souboru.

4 Uživatelská příručka (Příkazy)

Po spuštění aplikace se ocitnete v příkazové řádce `MyNodes:/Inode0>`. Zde je přehled dostupných příkazů.

4.1 Správa disku a systému

`format <velikost>` Naformátuje virtuální disk na zadanou velikost. Např. `format 100MB` nebo `format 104857600` (v bajtech).

`statfs` Zobrazí statistiky využití disku (volné inody, bloky, fragmentace).

`load <skript>` Načte textový soubor s příkazy a postupně je vykoná (vhodné pro testování).

`exit` Ukončí program.

4.2 Práce se soubory

`touch <název>` Vytvoří prázdný soubor.

`write <název> «text>` Zapiše text do souboru (přepíše existující obsah).

`cat <název>` Vypíše obsah souboru do terminálu.

`add <zdroj> <cíl>` Připojí obsah zdrojového souboru na konec cílového souboru.

`cp <zdroj> <cíl>` Zkopíruje soubor na nové umístění.

`mv <zdroj> <cíl>` Přesune nebo přejmenuje soubor.

`rm <název>` Smaže soubor a uvolní jeho datové bloky.

`info <název>` Zobrazí detailní metadata inody (velikost, použité bloky, typ adresace).

`xcp <f1> <f2> <cíl>` Sloučí obsah dvou souborů (`f1 + f2`) do nového souboru `cíl`.

4.3 Práce s adresáři

`ls [cesta]` Vypíše obsah aktuálního nebo zadaného adresáře.

`mkdir <název>` Vytvoří nový adresář.

`cd <cesta>` Změní aktuální pracovní adresář.

`rmdir <název>` Smaže prázdný adresář.

`pwd` Vypíše aktuální cestu.

4.4 Import a Export (Hostitelský systém)

`incp <host_file> <fs_file>` Importuje reálný soubor z hostitelského PC do virtuálního systému.

`outcp <fs_file> <host_file>` Exportuje soubor z našeho filesystemu zpět na hostitelské PC.

5 Příklad použití

Následující scénář demonstruje práci s naimplementovaným systémem. Ukazuje inicializaci disku, import externích dat, manipulaci se soubory pomocí přesunů a kopií, a finální kontrolu stavu souborového systému.

```
1 MyNodes:/Inode0> format 100MB
2 Formatting disk: data/filesystem.dat... Success.
3
4 MyNodes:/Inode0> mkdir users
5 MyNodes:/Inode0> cd users
6 MyNodes:/users> touch notes.txt
7 MyNodes:/users> write notes.txt "Project info..."
8
9 # Import 45MB souboru (navez musi byt <= 11 znaku)
10 MyNodes:/users> incp video.mp4 video.dat
11 Importing 'video.mp4' to 'video.dat'... Success.
12
13 MyNodes:/users> info video.dat
14 ===== File Stats: video.dat =====
15 Inode ID:      6
16 Size:          47185920 bytes
17 -- Block Usage --
18 Direct Blocks: [ 12, ... 21 ] (Full)
19 Single Ind.:   22 (Table holds 1024 blocks)
20 Double Ind.:   1050 (Master Table -> Inner Tables)
21 Total Blocks:  11520
22 =====
23
24 # Kopie a Presun (notes.txt -> notes.bak)
25 MyNodes:/users> cp notes.txt notes.bak
26 MyNodes:/users> mv video.dat /final.dat
27 Moved 'users/video.dat' to '/final.dat'.
28
29 MyNodes:/users> cd /
30 MyNodes:/Inode0> outcp notes.bak C:/Export/notes.txt
31 Exporting 'notes.bak'... Success.
32
33 MyNodes:/Inode0> rm final.dat
34 File 'final.dat' deleted.
35
36 MyNodes:/Inode0> statfs
37 Disk Name:      data/filesystem.dat
38 Inodes:         196 free / 200 total
39 Data Blocks:    25587 free / 25600 total
```

Listing 1: Pseudo ukázka relace v terminálu

6 Závěr

Tento projekt úspěšně realizoval kompletní simulaci souborového systému, přičemž největším přínosem bylo praktické osvojení práce s inodami. Do hloubky jsme pochopili jejich vnitřní strukturu a klíčovou roli při správě metadat, kdy jediná inoda o fixní velikosti dokáže efektivně adresovat data od malých textových souborů až po gigabytové objemy pomocí mechanismu přímých a nepřímých odkazů.

Dále jsme si prohloubili znalosti nízkoúrovňového programování v C++, zejména práci s binárními proudy a serializací struktur přímo na disk. Implementace alokačních bitmap a mechanismů pro ošetření kritických stavů vyústila v robustní a stabilní systém, který nám poskytl cenný vhled do toho, jak operační systémy efektivně spravují data.